

## TECHNICAL FILE-BY-FILE REPORT

# Aaj Kaunse Files Bane, Aur Kyun

Har naye file/folder ka purpose, banane ke steps, use hui logic, aur purane code mein kya-kya badlav hua uski wajah se - sab kuch ek jagah.

Prepared for: Team Reference

Date: 31 August 2026

Branch: feature/dev

8

NAYE FILES/FOLDERS

5

PURANE FILES MEIN BADLAV

815

LINES ADD HUI (GIT DIFF SE)

# Sab kuch, ek table mein

Yeh saari list `git diff` se nikaali gayi hai - guess ya yaadasht se nahi, isliye 100% accurate hai.

## Naye files/folders (8)

| FILE                                                  | EK LINE MEIN PURPOSE                                          |
|-------------------------------------------------------|---------------------------------------------------------------|
| <code>strategy-feed/server.py</code>                  | Python TCP server - bahar ke algo systems se connect hota hai |
| <code>strategy-feed/requirements.txt</code>           | Python dependencies ki list                                   |
| <code>strategy-feed/.env.example</code>               | Secret config ka template                                     |
| <code>server/src/routes/strategyFeed.routes.js</code> | Naye API endpoints - data andar/bahar                         |
| <code>client/src/hooks/useStrategyFeed.js</code>      | React hook - live updates sunne ke liye                       |
| <code>.github/workflows/ci.yml</code>                 | Har push par automatic check chalata hai                      |
| <code>server/scripts/load-test-orders.js</code>       | System ko load ke saath test karne ka tool                    |
| <code>docs/REDUNDANCY_AND_FAILOVER.md</code>          | Database redundancy ki planning documentation                 |

## Purane files mein badlav (5)

| FILE                                                 | EK LINE MEIN KYA BADLA                                                    |
|------------------------------------------------------|---------------------------------------------------------------------------|
| <code>server/src/index.js</code>                     | Naya route register kiya (2 lines)                                        |
| <code>server/src/services/wsHub.service.js</code>    | Naya "broadcast to everyone" function add hua                             |
| <code>client/src/components/StrategyPanel.jsx</code> | Deploy/Stop button ab asli backend call karta hai + live feed dikhata hai |
| <code>server/.env.example</code>                     | 2 naye config variables add hue                                           |
| <code>docs/BACKUP_AND_RETENTION.md</code>            | Real cron schedule + real restore-proof add hua                           |

## Yeh 5 files ek saath kaam karti hain

Yeh aapke senior ke ask ka jawab hai - "outside algo/strategist systems se connect karo, unka data dikhao, aur unhe command bhi bhej sako." Isko banane ke liye 5 files chahiye hui - 3 Python side par, 2 Node backend side par, plus frontend mein 1 hook.

### strategy-feed/server.py

NAYA

#### KYA HAI

Python mein likha ek TCP server - yeh literally woh cheez hai jo senior ne ask ki thi.

#### KYUN BANAYA

Bahar ke algo/strategist systems ko ek jagah chahiye thi jahan woh apna live signal bhej sakein, aur jahan se unhe command (Deploy/Stop) mil sake.

#### STEPS JO FOLLOW KIYE

- `asyncio` use kiya - ek saath multiple systems connect ho sakein, ek ke wait mein doosra na ruke
- Protocol simple rakha: har message ek line, JSON format mein - kisi bhi language se bhejna easy
- Ek alag HTTP "control port" (9101) bhi banaya - backend se ulta command bhejne ke liye
- Pehle terminal mein secret type karna padta tha - baad mein `python-dotenv` add kiya taaki secret `.env` file se load ho, kabhi terminal history mein na jaaye

#### LOGIC

Do independent direction hain isi file mein: (1) TCP se data aata hai → console par print → backend ko forward. (2) Backend se command aata hai (HTTP) → jis TCP client se connected hai use likh diya jaata hai. Dono ek hi process mein saath-saath chalte hain, ek dusre ko block nahi karte.

### strategy-feed/requirements.txt + .env.example

NAYA

#### KYA HAI

`requirements.txt` mein Python packages ki list (`aiohttp` - HTTP calls ke liye, `python-dotenv` - secret load karne ke liye). `.env.example` ek template hai jo dikhata hai kya-kya config chahiye, real secret ke bina.

#### KYUN BANAYA

Har naye Python project ki tarah - dependencies aur config ka ek standard, safe tareeka chahiye tha.

## Backend aur frontend ka hissa

server/src/routes/strategyFeed.routes.js

NAYA

### KYA HAI

2 naye API endpoints: `POST /api/strategy-feed/ingest` (Python se data andar aata hai) aur `POST /api/strategy-feed/command` (trader se command bahar jaata hai).

### STEPS JO FOLLOW KIYE

- `/ingest` : normal login se protect nahi kiya (kyunki yeh Python server se aata hai, kisi insaan se nahi) - iske bajaye ek shared secret header check kiya
- `/command` : normal trader login se protect kiya ( `authenticate` + `authorize('TRADER')` ), same pattern jo order-placement mein pehle se use ho raha hai

### LOGIC (EK IMPORTANT DETAIL)

Secret check mein: agar server par secret set hi nahi hai, to request ko reject karna hai, allow nahi. Yeh isliye zaroori hai kyunki agar galti se secret ka env variable set hona bhool jaaye, aur code sirf "header === secret" check kare, to dono undefined ho jaate aur match ho jaata - matlab bina kisi secret ke koi bhi data bhej sakta. Isliye pehle check hai "secret set hai ya nahi", tabhi match dekha jaata hai.

client/src/hooks/useStrategyFeed.js

NAYA

### KYA HAI

Ek React hook jo WebSocket se live updates sunta hai, jab bhi Strategy Panel khula ho.

### STEPS JO FOLLOW KIYE

Naya pattern nahi banaya - pehle se ek similar hook tha ( `useOrderUpdates.js` , orders/fills ke liye), usi ka structure copy kiya: ek shared WebSocket connection, jo sirf tab khulti hai jab koi component use kar raha ho, aur band ho jaati hai jab koi nahi.

### LOGIC

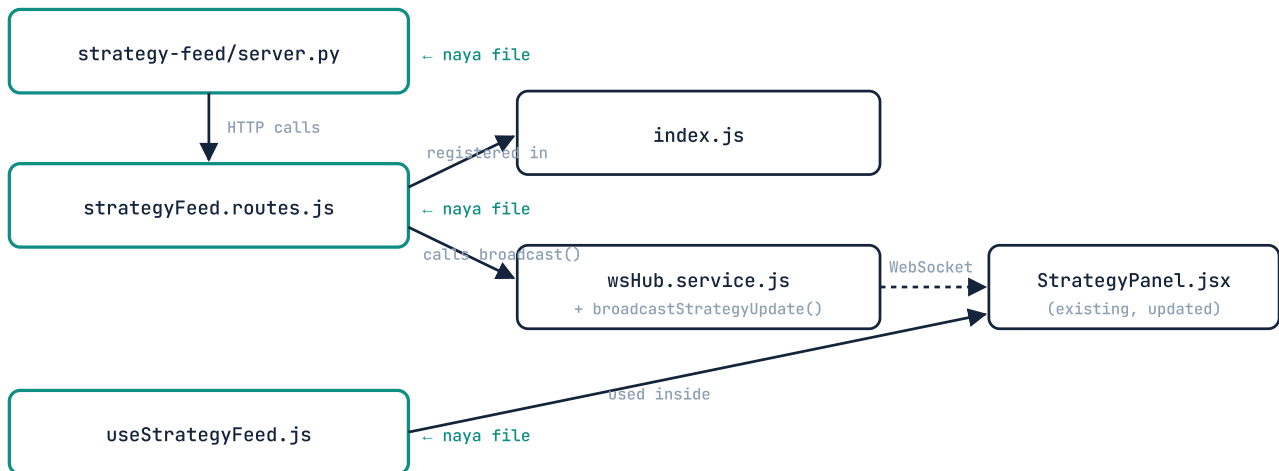
Aane wale message ka `type` field check karta hai - sirf `"STRATEGY_FEED_UPDATE"` wale messages ko aage bhejta hai, baaki (market ticks, order updates) ignore karta hai. Isse ek hi WebSocket connection kai alag-alag cheezon ke liye reuse ho jaata hai.

### YEH SAB EK DUSRE SE CONNECTED KAISE HAIN

Agla page (Section 03) mein ek diagram hai jo dikhata hai in files ka aapas mein connection, aur purane files mein kya badlav hua isi wajah se.

## Naye files ne purane code ko kaise touch kiya

Koi bhi naya feature hawa mein nahi chalta - use existing app se connect karna padta hai. Yahan exactly wahi 3 jagah hain jahan purana code badla, aur kis naye file ki wajah se.



Teal boxes = aaj ke naye files. Safed boxes = purane files jinme badlav hua, kyunki inhe naye files se connect karna zaroori tha.

### server/src/index.js

BADLA

**Kya badla:** 2 lines add hui - `import strategyFeedRoutes` aur `app.use('/api/strategy-feed', ...)`

**Kis wajah se:** `strategyFeed.routes.js` naya file bana - koi bhi naya route file bina register kiye kaam nahi karta

### server/src/services/wsHub.service.js

BADLA

**Kya badla:** Naya function `broadcastStrategyUpdate()` add hua - message ko HAR connected user ko bhejta hai

**Kis wajah se:** `strategyFeed.routes.js` ko is function ki zarurat thi - pehle se jo functions the woh sirf ek specific user ko bhejte the, "sabko bhejo" wala function nahi tha

### client/src/components/StrategyPanel.jsx

BADLA

**Kya badla:** Deploy/Stop button ab real API call karta hai (pehle sirf local state change hota tha), plus naya "External Feed" section

**Kis wajah se:** `useStrategyFeed.js` hook aur naya `/command` endpoint - dono ka use yahin hua

### server/.env.example

BADLA

**Kya badla:** `STRATEGY_FEED_SECRET` aur `STRATEGY_FEED_CONTROL_URL` add hue

**Kis wajah se:** Python server aur Node backend ke beech secret match hona chahiye, aur backend ko Python ka control-port URL pata hona chahiye

## Yeh teeno independent tools hain – app ke andar import nahi hote

In teeno ka app ke chalte hue code se koi seedha connection nahi hai - inhe "tools" ki tarah samjho, jo app ke bahar se app ko check/verify/document karte hain.

`.github/workflows/ci.yml`

NAYA

### KYA HAI

GitHub Actions ka CI/CD pipeline - har push par automatically 2 jobs chalata hai.

### KYUN BANAYA

Pehle koi automatic check nahi tha. Agar galti wala code push ho jaaye, kisi ko pata nahi chalta jab tak manually test na kare.

### STEPS

- "client" job: lint check + production build
- "server" job: ek throwaway Postgres database spin up karta hai (GitHub ke apne feature se), saare migrations run karta hai, phir asli server ko us par boot karke `/api/health` check karta hai

### LOGIC

Dummy connection string se test nahi kiya - kyunki server startup par hi ek real database query chalti hai (`refreshAllRmsCaches`). Isliye ek asli, fresh Postgres chahiye tha CI mein bhi, warna test jhoothi confidence deta.

`server/scripts/load-test-orders.js`

NAYA

### KYA HAI

Ek tool jo order-placement API ko bahut fast, bahut saare requests bhejke test karta hai.

### KYUN BANAYA

NSE ka rule hai max 120 orders/second per trader - lekin kabhi test nahi kiya tha ki asal load mein system kya karta hai.

### STEPS

- Disposable (use-and-throw) test trader accounts banata hai
- Un accounts ke liye seedha JWT token banata hai (login process se guzre bina, taaki test fast chale)
- Ek saath bahut saare orders bhejta hai, result record karta hai (kitne accept/reject hue, latency kitni thi)
- Test ke baad apna banaya hua saara data khud delete kar deta hai

### LOGIC - AUR EK ZAROORI BAAT

Pehle version mein ek badi galti thi - agar kaafi trader accounts already the, to woh unhi ko reuse kar leta tha testing ke liye. Cleanup step phir us account ka SAARA data delete kar deta - test data ke saath-saath real data bhi. Yeh asal mein hua bhi (Section 05 dekho). Fix karne ke baad, ab yeh **hamesha naye disposable accounts** hi banata

hai, aur cleanup se pehle ek hard check karta hai - agar list mein koi bhi account "disposable" na ho, to poora cleanup hi cancel ho jaata hai. Matlab ab yeh galti dobara ho hi nahi sakti.

docs/REDUNDANCY\_AND\_FAILOVER.md

NAYA

#### KYA HAI

Documentation - agar database server hi crash ho jaaye to kya hoga, uski planning.

#### KYUN BANAYA

Aapne pucha "database backup aur redundancy kaise fix karein" - backup to already tha, lekin redundancy (zero-downtime failover) bilkul nahi tha.

#### LOGIC - KYUN SIRF DOCUMENTATION, CODE NAHI

Redundancy banane ke do rasta hain: (a) cloud database (jaise AWS RDS) use karo - yeh sirf ek setting/toggle jitna simple hai, (b) khud host karo - iske liye Patroni/repmgr jaisi tooling chahiye, jo kaafi zyada kaam hai. Konsa rasta lena hai, yeh depend karta hai ek decision par jo abhi tak hua hi nahi - deployment kahan hoga. Aapne khud kaha "abhi sirf plan likho, kuch build mat karo" - kyunki galat rasta pehle build karna time waste ho sakta tha.



## Ek galti hui thi, isliye poora sach yahan hai

`load-test-orders.js` banate waqt, uske pehle version ne ek real trader account ( `Trader1` ) ko test ke liye reuse kar liya. Test khatam hone ke baad cleanup step ne us account ka SAARA data delete kar diya - jisme 2 asli, real orders bhi the (test data nahi).

### Kaise pakda gaya, aur kaise theek kiya

1. Routine verification ke dauraan ek row count match nahi hua - turant check kiya, assume nahi kiya ki sab theek hai
2. Confirm hua ki 2 real orders (aur unki fills, history, audit trail) delete ho gaye the
3. Nightly backup se saare 18 rows wapas restore kiye - bilkul same ID, same timestamp, same values ke saath, verify karke
4. Tool ko fix kiya - ab yeh sirf disposable accounts banata hai, aur ek hard safety-check hai jo kabhi bhi real account ko touch hone hi nahi deta

#### YEH HONESTLY KYUN BATAYA

Galti chhupane se koi fayda nahi - aur is incident ne asal mein yeh bhi prove kiya ki backup system genuinely kaam karta hai, sirf theory mein nahi.

# Kuch changes jo git mein track nahi hote

| KYA                | KYA KIYA                                               | KYUN GIT MEIN NAHI HAI                                           |
|--------------------|--------------------------------------------------------|------------------------------------------------------------------|
| Backup ka cron job | Machine ke crontab mein daily 2am ka schedule add kiya | System-level setting hai, repo file nahi                         |
| strategy-feed/.env | Real secret value ke saath actual config file banayi   | Yeh secret hai - .gitignore isko jaan-boojhkar exclude karta hai |

iwmQT Trading Terminal — Technical File-by-File Report — 31 August 2026 — branch feature/dev . Saari file list git diff se verify ki gayi hai.